

Lab 2: Database Constraint Design (sqlite3)

This notebook implements only the required tasks:

- Section 2 (Intermediate): Task 1 and Task 3
- Section 3 (Advanced): Task 1 and Task 2

Implementation base: Python + sqlite3.

```
In [ ]: '''
        定义数据库的基本接口
        '''

import sqlite3
from textwrap import dedent

conn = sqlite3.connect(":memory:")
conn.row_factory = sqlite3.Row
conn.execute("PRAGMA foreign_keys = ON;")

def show(sql, params=()):
    rows = conn.execute(sql, params).fetchall()
    print(f"{len(rows)} row(s)")
    for r in rows:
        print(dict(r))

def run(sql, params=()):
    conn.execute(sql, params)
    conn.commit()

def run_script(sql_script):
    conn.executescript(sql_script)
    conn.commit()

def expect_fail(sql, params=()):
    try:
        conn.execute(sql, params).fetchall()
        conn.commit()
        print("FAILED: expected an error but succeeded")
    except Exception as e:
        conn.rollback()
        print("Expected failure:", e)
```

1) 数据库表定义

基础功能实现

1. eno和dno是递增序列号形式的主码，长度为4的整型

```
dno TEXT PRIMARY KEY
      CHECK (dno GLOB '[0-9][0-9][0-9][0-9]' AND dno BETWEEN
'0001' AND '9999'),

eno TEXT PRIMARY KEY
      CHECK (eno GLOB '[0-9][0-9][0-9][0-9]' AND eno BETWEEN
'0001' AND '9999'),
```

2. Emp中的dno为参照Dept的外码， Dept的manager为参照Emp的外码

```
dno TEXT NOT NULL,
      FOREIGN KEY (dno) REFERENCES Dept(dno)
      ON UPDATE CASCADE
      ON DELETE RESTRICT
      DEFERRABLE INITIALLY DEFERRED,
```

3. 测试外码定义的三种形式 见末尾

4. 限定dname为枚举型（数学学院、计算机学院、智能学院、电子学院、元培学院）

```
dname TEXT NOT NULL CHECK (dname IN ('Mathematics', 'ComputerScience',
'AI', 'Electronics', 'Yuanpei')),
```

5. 限定position为枚举型（教师、教务、会计、秘书） position TEXT NOT NULL

```
CHECK (position IN ('Teacher', 'AcademicAffairs', 'Accountant',
'Secretary'))
```

6. 限定level为1到5，缺省为3，salary为2000~200000 level INTEGER NOT NULL

```
DEFAULT 3 CHECK (level BETWEEN 1 AND 5)
```

中级约束设计

1. 将salary划分为5个区间，每个区间对应一个level值，保证每个员工的工资值和他的level值是正确对应的，这属于行级约束

```
CHECK (
      (salary BETWEEN 2000 AND 5000 AND level = 1) OR
      (salary > 5000 AND salary <= 10000 AND level = 2) OR
      (salary > 10000 AND salary <= 20000 AND level = 3) OR
      (salary > 20000 AND salary <= 50000 AND level = 4) OR
      (salary > 50000 AND salary <= 200000 AND level = 5)
)
```

3. 编写一个函数，它接收一个身份证号的前17位，生成一个身份证的校验码 见下文

高级约束设计 (trigger)

1. 管理者的工资必须高于他所管理的任何一个员工.

- `trg_emp_mgr_check_bi`：拦“插入员工”时的违规

- `trg_emp_mgr_check_bu` : 拦“修改员工工资/部门”时的违规
- `trg_mgr_raise_check` : 拦“修改经理工资”时的违规
- `trg_dept_mgr_check_bu` : 拦“修改部门经理”时的违规

2. 任何一个员工工资的变化额度，都应该体现在他所在部门的预算上面

```
In [ ]: schema_sql = dedent("""
CREATE TABLE Dept (
    dno TEXT PRIMARY KEY
        CHECK (dno GLOB '[0-9][0-9][0-9][0-9]' AND dno BETWEEN '0001' AND '9999'
    )
    , dname TEXT NOT NULL CHECK (dname IN ('Mathematics','ComputerScience','AI','E
    budget REAL NOT NULL DEFAULT 0 CHECK (budget >= 0),
    manager TEXT UNIQUE,
    FOREIGN KEY (manager) REFERENCES Emp(eno)
        ON DELETE SET NULL
        DEFERRABLE INITIALLY DEFERRED
);

CREATE TABLE Emp (
    eno TEXT PRIMARY KEY
        CHECK (eno GLOB '[0-9][0-9][0-9][0-9]' AND eno BETWEEN '0001' AND '9999'
    )
    , ename TEXT NOT NULL,
    ID_number TEXT NOT NULL CHECK (length(ID_number) = 18),
    level INTEGER NOT NULL DEFAULT 3 CHECK (level BETWEEN 1 AND 5),
    position TEXT NOT NULL CHECK (position IN ('Teacher','AcademicAffairs','Acco
    salary REAL NOT NULL CHECK (salary BETWEEN 2000 AND 200000),
    dno TEXT NOT NULL,
    FOREIGN KEY (dno) REFERENCES Dept(dno)
        ON UPDATE CASCADE
        ON DELETE RESTRICT
        DEFERRABLE INITIALLY DEFERRED,

    CHECK (
        (salary BETWEEN 2000 AND 5000 AND level = 1) OR
        (salary > 5000 AND salary <= 10000 AND level = 2) OR
        (salary > 10000 AND salary <= 20000 AND level = 3) OR
        (salary > 20000 AND salary <= 50000 AND level = 4) OR
        (salary > 50000 AND salary <= 200000 AND level = 5)
    )
);

-- Enforce sequence style IDs directly at declaration/insert constraint layer.
CREATE TRIGGER trg_dept_dno_sequence
BEFORE INSERT ON Dept
BEGIN
    SELECT CASE
        WHEN NEW.dno IS NULL
        THEN RAISE(ABORT, 'dno must be provided as a 4-digit code')
        WHEN NEW.dno NOT GLOB '[0-9][0-9][0-9][0-9]' OR NEW.dno < '0001' OR NEW.
        THEN RAISE(ABORT, 'dno must be a 4-digit code from 0001 to 9999')
        WHEN CAST(NEW.dno AS INTEGER) <> IFNULL((SELECT MAX(CAST(dno AS INTEGER)
        THEN RAISE(ABORT, 'dno must be strictly increasing (max + 1)')
    END;
END;

CREATE TRIGGER trg_emp_eno_sequence
BEFORE INSERT ON Emp
BEGIN
```

```

        SELECT CASE
            WHEN NEW.eno IS NULL
            THEN RAISE(ABORT, 'eno must be provided as a 4-digit code')
            WHEN NEW.eno NOT GLOB '[0-9][0-9][0-9][0-9]' OR NEW.eno < '0001' OR NEW.
            THEN RAISE(ABORT, 'eno must be a 4-digit code from 0001 to 9999')
            WHEN CAST(NEW.eno AS INTEGER) <> IFNULL((SELECT MAX(CAST(eno AS INTEGER)
            THEN RAISE(ABORT, 'eno must be strictly increasing (max + 1)')
        END;
    END;

CREATE TRIGGER trg_dept_dno_immutable
BEFORE UPDATE OF dno ON Dept
BEGIN
    SELECT RAISE(ABORT, 'dno is immutable');
END;

CREATE TRIGGER trg_emp_eno_immutable
BEFORE UPDATE OF eno ON Emp
BEGIN
    SELECT RAISE(ABORT, 'eno is immutable');
END;

-- Advanced Task 2: budget maintenance
CREATE TRIGGER trg_emp_budget_ai
AFTER INSERT ON Emp
BEGIN
    UPDATE Dept
        SET budget = budget + NEW.salary
        WHERE dno = NEW.dno;
END;

CREATE TRIGGER trg_emp_budget_ad
AFTER DELETE ON Emp
BEGIN
    UPDATE Dept
        SET budget = budget - OLD.salary
        WHERE dno = OLD.dno;
END;

CREATE TRIGGER trg_emp_budget_au
AFTER UPDATE OF salary, dno ON Emp
BEGIN
    UPDATE Dept
        SET budget = budget - OLD.salary
        WHERE dno = OLD.dno;

    UPDATE Dept
        SET budget = budget + NEW.salary
        WHERE dno = NEW.dno;
END;

-- Advanced Task 1: non-manager employee must be paid less than department manager
CREATE TRIGGER trg_emp_mgr_check_bi
BEFORE INSERT ON Emp
BEGIN
    SELECT CASE
        WHEN EXISTS (
            SELECT 1

```

```

        FROM Dept d
        JOIN Emp m ON m.eno = d.manager
WHERE d.dno = NEW.dno
      AND d.manager IS NOT NULL
      AND NEW.eno <> d.manager
      AND NEW.salary >= m.salary
    )
    THEN RAISE(ABORT, 'Employee salary must be lower than manager salary')
END;
END;

CREATE TRIGGER trg_emp_mgr_check_bu
BEFORE UPDATE OF salary, dno ON Emp
BEGIN
    SELECT CASE
        WHEN EXISTS (
            SELECT 1
            FROM Dept d
            JOIN Emp m ON m.eno = d.manager
            WHERE d.dno = NEW.dno
                  AND d.manager IS NOT NULL
                  AND NEW.eno <> d.manager
                  AND NEW.salary >= m.salary
        )
        THEN RAISE(ABORT, 'Employee salary must be lower than manager salary')
    END;
END;

-- Advanced Task 1: manager salary must remain above all subordinates
CREATE TRIGGER trg_mgr_raise_check
BEFORE UPDATE OF salary ON Emp
WHEN EXISTS (SELECT 1 FROM Dept d WHERE d.manager = NEW.eno)
BEGIN
    SELECT CASE
        WHEN EXISTS (
            SELECT 1
            FROM Dept d
            JOIN Emp e ON e.dno = d.dno
            WHERE d.manager = NEW.eno
                  AND e.eno <> NEW.eno
                  AND e.salary >= NEW.salary
        )
        THEN RAISE(ABORT, 'Manager salary must be higher than all subordinates')
    END;
END;

-- Advanced Task 1: manager assignment checks
CREATE TRIGGER trg_dept_mgr_check_bu
BEFORE UPDATE OF manager ON Dept
WHEN NEW.manager IS NOT NULL
BEGIN
    SELECT CASE
        WHEN EXISTS (
            SELECT 1
            FROM Emp m
            WHERE m.eno = NEW.manager
                  AND m.dno <> NEW.dno
        )
        THEN RAISE(ABORT, 'Manager must belong to the same department')
    END;
END;

```

```

        SELECT CASE
            WHEN EXISTS (
                SELECT 1
                FROM Emp e
                JOIN Emp m ON m.eno = NEW.manager
                WHERE e.dno = NEW.dno
                AND e.eno <> NEW.manager
                AND e.salary >= m.salary
            )
            THEN RAISE(ABORT, 'Manager salary must be higher than all subordinates')
        END;
    END;
    """

run_script(schema_sql)
print('Schema and triggers created.')

```

Schema and triggers created.

2) Seed Data + Intermediate Task 1 Tests

```

In [ ]: run("INSERT INTO Dept(dno, dname) VALUES ('0001', 'ComputerScience')")
run("INSERT INTO Dept(dno, dname) VALUES ('0002', 'Mathematics')")

print('Negative test: dno must be 4 digits (should fail)')
expect_fail("INSERT INTO Dept(dno, dname) VALUES ('003', 'AI')")

print('Negative test: dno must be strictly increasing (should fail)')
expect_fail("INSERT INTO Dept(dno, dname) VALUES ('0004', 'AI')")

run("""
INSERT INTO Emp(eno, ename, ID_number, level, position, salary, dno)
VALUES ('0001', 'Manager_A', '110105197001010013', 5, 'Teacher', 90000, '0001')
""")
run("""
INSERT INTO Emp(eno, ename, ID_number, level, position, salary, dno)
VALUES ('0002', 'Employee_B', '110105198802020024', 4, 'Teacher', 40000, '0001')
""")
run("""
INSERT INTO Emp(eno, ename, ID_number, level, position, salary, dno)
VALUES ('0003', 'Employee_C', '110105199303030035', 3, 'AcademicAffairs', 15000,
""")

run("UPDATE Dept SET manager = '0001' WHERE dno = '0001'")

print('Current employees:')
show("SELECT eno, ename, level, salary, dno FROM Emp ORDER BY eno")

print('Current departments:')
show("SELECT dno, dname, manager, budget FROM Dept ORDER BY dno")

print('Negative test: salary-level mismatch should fail')
expect_fail("""
INSERT INTO Emp(eno, ename, ID_number, level, position, salary, dno)
VALUES ('0004', 'Invalid_Level', '110105199901010046', 1, 'Secretary', 12000, '0
""")

```

Negative test: dno must be 4 digits (should fail)
 Expected failure: dno must be a 4-digit code from 0001 to 9999
 Negative test: dno must be strictly increasing (should fail)
 Expected failure: dno must be strictly increasing (max + 1)
 Current employees:
 3 row(s)
 {'eno': '0001', 'ename': 'Manager_A', 'level': 5, 'salary': 90000.0, 'dno': '0001'}
 {'eno': '0002', 'ename': 'Employee_B', 'level': 4, 'salary': 40000.0, 'dno': '0001'}
 {'eno': '0003', 'ename': 'Employee_C', 'level': 3, 'salary': 15000.0, 'dno': '0001'}
 Current departments:
 2 row(s)
 {'dno': '0001', 'dname': 'ComputerScience', 'manager': '0001', 'budget': 145000.0}
 {'dno': '0002', 'dname': 'Mathematics', 'manager': None, 'budget': 0.0}
 Negative test: salary-level mismatch should fail
 Expected failure: CHECK constraint failed: (salary BETWEEN 2000 AND 5000 AND level = 1) OR
 (salary > 5000 AND salary <= 10000 AND level = 2) OR
 (salary > 10000 AND salary <= 20000 AND level = 3) OR
 (salary > 20000 AND salary <= 50000 AND level = 4) OR
 (salary > 50000 AND salary <= 200000 AND level = 5)

1.3 Three Definition Forms of Foreign Key (Positive + Negative)

This block tests three FK declaration forms in SQLite:

1. Column-level REFERENCES
2. Table-level FOREIGN KEY (...) REFERENCES ...
3. Named FK constraint with referential actions (ON DELETE CASCADE)

```
In [ ]: run_script("""
DROP TABLE IF EXISTS fk3_child_named;
DROP TABLE IF EXISTS fk3_child_table;
DROP TABLE IF EXISTS fk3_child_column;
DROP TABLE IF EXISTS fk3_parent;

CREATE TABLE fk3_parent (
    pid TEXT PRIMARY KEY CHECK (pid GLOB '[0-9][0-9][0-9][0-9]')
);

-- Form 1: column-level foreign key
CREATE TABLE fk3_child_column (
    cid INTEGER PRIMARY KEY,
    pid TEXT REFERENCES fk3_parent(pid)
);

-- Form 2: table-level (unnamed) foreign key
CREATE TABLE fk3_child_table (
    cid INTEGER PRIMARY KEY,
    pid TEXT,
    FOREIGN KEY (pid) REFERENCES fk3_parent(pid)
);

-- Form 3: table-level named foreign key with action
```

```

CREATE TABLE fk3_child_named (
    cid INTEGER PRIMARY KEY,
    pid TEXT,
    CONSTRAINT fk3_named_fk
        FOREIGN KEY (pid) REFERENCES fk3_parent(pid)
        ON DELETE CASCADE
);
""")

run("INSERT INTO fk3_parent(pid) VALUES ('0001')")
run("INSERT INTO fk3_parent(pid) VALUES ('0002')")

print('Positive tests: valid parent key should succeed in all three forms')
run("INSERT INTO fk3_child_column(cid, pid) VALUES (1, '0001')")
run("INSERT INTO fk3_child_table(cid, pid) VALUES (1, '0001')")
run("INSERT INTO fk3_child_named(cid, pid) VALUES (1, '0001')")
show("SELECT * FROM fk3_child_column ORDER BY cid")
show("SELECT * FROM fk3_child_table ORDER BY cid")
show("SELECT * FROM fk3_child_named ORDER BY cid")

print('Negative tests: missing parent key should fail in all three forms')
expect_fail("INSERT INTO fk3_child_column(cid, pid) VALUES (2, '9999')")
expect_fail("INSERT INTO fk3_child_table(cid, pid) VALUES (2, '9999')")
expect_fail("INSERT INTO fk3_child_named(cid, pid) VALUES (2, '9999')")

print('Action test for form 3: ON DELETE CASCADE')
run("INSERT INTO fk3_child_named(cid, pid) VALUES (3, '0002')")
show("SELECT * FROM fk3_child_named ORDER BY cid")
run("DELETE FROM fk3_parent WHERE pid = '0002'")
show("SELECT * FROM fk3_child_named ORDER BY cid")

```

Positive tests: valid parent key should succeed in all three forms

```

1 row(s)
{'cid': 1, 'pid': '0001'}
1 row(s)
{'cid': 1, 'pid': '0001'}
1 row(s)
{'cid': 1, 'pid': '0001'}

```

Negative tests: missing parent key should fail in all three forms

```

Expected failure: FOREIGN KEY constraint failed
Expected failure: FOREIGN KEY constraint failed
Expected failure: FOREIGN KEY constraint failed

```

Action test for form 3: ON DELETE CASCADE

```

2 row(s)
{'cid': 1, 'pid': '0001'}
{'cid': 3, 'pid': '0002'}
1 row(s)
{'cid': 1, 'pid': '0001'}

```

3) Intermediate Task 3: ID Checksum Function

Register Python function as SQLite function `id_checksum`.

```

In [ ]: def id_checksum(id17: str) -> str:
        s = str(id17).strip().upper()
        if len(s) != 17 or (not s.isdigit()):
            raise ValueError('Input must be exactly 17 digits')

        weights = [7, 9, 10, 5, 8, 4, 2, 1, 6, 3, 7, 9, 10, 5, 8, 4, 2]

```



```

mapping = ['1', '0', 'X', '9', '8', '7', '6', '5', '4', '3', '2']
total = sum(int(ch) * w for ch, w in zip(s, weights))
return mapping[total % 11]

conn.create_function('id_checksum', 1, id_checksum)

print('Example: 11010519491231002 -> expected check code X')
show("SELECT '11010519491231002' AS id17, id_checksum('11010519491231002') AS ch

print('Negative test: invalid input length should fail')
expect_fail("SELECT id_checksum('12345')")

```

Example: 11010519491231002 -> expected check code X
 1 row(s)
 {'id17': '11010519491231002', 'check_code': 'X'}
 Negative test: invalid input length should fail
 Expected failure: user-defined function raised exception

4) Advanced Task 1: Manager Salary Constraint Tests

```

In [ ]: print('Negative test: employee salary >= manager salary should fail')
expect_fail("""
INSERT INTO Emp(eno, ename, ID_number, level, position, salary, dno)
VALUES ('0004', 'High_Paid_Staff', '110105199404040047', 5, 'Teacher', 95000, '0
""")

print('Negative test: lowering manager salary below subordinate should fail')
expect_fail("UPDATE Emp SET salary = 35000, level = 4 WHERE eno = '0001'")

run("""
INSERT INTO Emp(eno, ename, ID_number, level, position, salary, dno)
VALUES ('0004', 'Staff_D', '110105198707070058', 4, 'Accountant', 30000, '0002')
""")
run("UPDATE Emp SET dno = '0002' WHERE eno = '0002'")

print('Negative test: assigning underpaid manager should fail')
expect_fail("UPDATE Dept SET manager = '0004' WHERE dno = '0002'")

print('Positive test: assign a qualified manager')
run("""
INSERT INTO Emp(eno, ename, ID_number, level, position, salary, dno)
VALUES ('0005', 'Manager_E', '110105197505050069', 5, 'Teacher', 70000, '0002')
""")
run("UPDATE Dept SET manager = '0005' WHERE dno = '0002'")
show("SELECT dno, dname, manager, budget FROM Dept ORDER BY dno")

```

Negative test: employee salary >= manager salary should fail
 Expected failure: Employee salary must be lower than manager salary
 Negative test: lowering manager salary below subordinate should fail
 Expected failure: Manager salary must be higher than all subordinates
 Negative test: assigning underpaid manager should fail
 Expected failure: Manager salary must be higher than all subordinates
 Positive test: assign a qualified manager
 2 row(s)
 {'dno': '0001', 'dname': 'ComputerScience', 'manager': '0001', 'budget': 105000.
 0}
 {'dno': '0002', 'dname': 'Mathematics', 'manager': '0005', 'budget': 140000.0}

5) Advanced Task 2: Incremental Budget Maintenance Tests

```
In [ ]: print('Budgets before updates:')
        show("SELECT dno, budget FROM Dept ORDER BY dno")

        print('Case 1: salary update 15000 -> 18000 (still level 3)')
        run("UPDATE Emp SET salary = 18000 WHERE eno = '0003'")
        show("SELECT dno, budget FROM Dept ORDER BY dno")

        print('Case 2: transfer + salary update 40000 -> 45000, dno 0002 -> 0001')
        run("UPDATE Emp SET dno = '0001', salary = 45000, level = 4 WHERE eno = '0002'")
        show("SELECT dno, budget FROM Dept ORDER BY dno")

        print('Case 3: raise to 80000 (still below manager 90000)')
        run("UPDATE Emp SET salary = 80000, level = 5 WHERE eno = '0002'")
        show("SELECT eno, ename, salary, dno FROM Emp WHERE eno IN ('0001','0002') ORDER
        show("SELECT dno, budget FROM Dept ORDER BY dno")

        print('Case 4 negative: raise to 95000 >= manager 90000 should fail')
        expect_fail("UPDATE Emp SET salary = 95000, level = 5 WHERE eno = '0002'")
        show("SELECT eno, ename, salary, dno FROM Emp WHERE eno IN ('0001','0002') ORDER
        show("SELECT dno, budget FROM Dept ORDER BY dno")
```

Budgets before updates:

2 row(s)

{'dno': '0001', 'budget': 105000.0}

{'dno': '0002', 'budget': 140000.0}

Case 1: salary update 15000 -> 18000 (still level 3)

2 row(s)

{'dno': '0001', 'budget': 108000.0}

{'dno': '0002', 'budget': 140000.0}

Case 2: transfer + salary update 40000 -> 45000, dno 0002 -> 0001

2 row(s)

{'dno': '0001', 'budget': 153000.0}

{'dno': '0002', 'budget': 100000.0}

Case 3: raise to 80000 (still below manager 90000)

2 row(s)

{'eno': '0001', 'ename': 'Manager_A', 'salary': 90000.0, 'dno': '0001'}

{'eno': '0002', 'ename': 'Employee_B', 'salary': 80000.0, 'dno': '0001'}

2 row(s)

{'dno': '0001', 'budget': 188000.0}

{'dno': '0002', 'budget': 100000.0}

Case 4 negative: raise to 95000 >= manager 90000 should fail

Expected failure: Employee salary must be lower than manager salary

2 row(s)

{'eno': '0001', 'ename': 'Manager_A', 'salary': 90000.0, 'dno': '0001'}

{'eno': '0002', 'ename': 'Employee_B', 'salary': 80000.0, 'dno': '0001'}

2 row(s)

{'dno': '0001', 'budget': 188000.0}

{'dno': '0002', 'budget': 100000.0}

6) Final Validation

```
In [ ]: print('Emp:')
        show("SELECT eno, ename, level, salary, dno FROM Emp ORDER BY eno")
```

```

print('Dept:')
show("SELECT dno, dname, manager, budget FROM Dept ORDER BY dno")

print('Budget consistency check (Dept.budget vs SUM(Emp.salary))')
show("""
SELECT d.dno,
       d.budget AS budget_in_dept,
       IFNULL((SELECT SUM(e.salary) FROM Emp e WHERE e.dno = d.dno), 0) AS salary_sum
FROM Dept d
ORDER BY d.dno
""")

```

Emp:

5 row(s)

```

{'eno': '0001', 'ename': 'Manager_A', 'level': 5, 'salary': 90000.0, 'dno': '0001'}
{'eno': '0002', 'ename': 'Employee_B', 'level': 5, 'salary': 80000.0, 'dno': '0001'}
{'eno': '0003', 'ename': 'Employee_C', 'level': 3, 'salary': 18000.0, 'dno': '0001'}
{'eno': '0004', 'ename': 'Staff_D', 'level': 4, 'salary': 30000.0, 'dno': '0002'}
{'eno': '0005', 'ename': 'Manager_E', 'level': 5, 'salary': 70000.0, 'dno': '0002'}

```

Dept:

2 row(s)

```

{'dno': '0001', 'dname': 'ComputerScience', 'manager': '0001', 'budget': 188000.0}

```

```

{'dno': '0002', 'dname': 'Mathematics', 'manager': '0005', 'budget': 100000.0}

```

Budget consistency check (Dept.budget vs SUM(Emp.salary))

2 row(s)

```

{'dno': '0001', 'budget_in_dept': 188000.0, 'salary_sum': 188000.0}

```

```

{'dno': '0002', 'budget_in_dept': 100000.0, 'salary_sum': 100000.0}

```